Sudoku Solver Based on Forward Checking - Report

Introduction to the Project

The goal of this project was to create a smart program that can automatically solve Sudoku puzzles. Sudoku is a popular logic game where you have to fill a nine-by-nine grid with numbers so that each row, column, and smaller three-by-three box contains the numbers one through nine exactly once. To make the computer solve this, we used a specific programming technique called the Forward Checking algorithm. This report explains the five most important methods we created to make the whole program work.

Setting Up the Game: The setup(...) Method

The setup method is the very first step in our program. It prepares a completely empty Sudoku board before the user can start playing or before the computer can start solving. When we call this method, we tell it the size of the puzzle we want to create. For a normal Sudoku game, the size is given as three, which tells the program to create a standard board made of three-by-three blocks.

```
public void setup(int size1) {
    size = size1;
    int n = size * size;

    // Initialize the empty puzzle board
    puzzle = new int[n][n];

    // Initialize the domain list for all variables
    D = new ArrayList<ArrayList<Integer>>(n * n);

    // Populate the initial domains for each cell (1 to 9)
    for (int i = 0; i < n * n; i++) {
        ArrayList<Integer> domain = new ArrayList<Integer>();
        for (int val = 1; val <= 9; val++) {
            domain.add(val);
        }
        D.add(domain);
    }
}
```

Interacting with the Game: The setValue(...) Method

When a user clicks on the screen and wants to put a number into the puzzle themselves, the program uses the setValue method. This method needs three pieces of information to work: the column, the row, and the actual number the user wants to insert.

However, this method is smart and does not just put random numbers in. It checks if the number is between zero and nine. A normal Sudoku only uses numbers from one to nine, but we intentionally allow the user to input a zero as well. If the user types a zero, the program understands this as a command to delete the number that is currently in that square, making it empty again.

```java
public void setValue(int col, int row, int value) {
    // Ensure the selected position is within the bounds of the board
    if(col >= puzzle.length || row >= puzzle[0].length ){
        return;
    }
    // Ensure the value is within the legal Sudoku range (0-9)
    if (value >= 0 && value <= 9){
        puzzle[col][row] = value;
    }
}
```

Loading Full Games: The readInPuzzle(...) Method

Typing in numbers one by one to test a new puzzle can take a lot of time. To make this process much faster and easier, we created the readInPuzzle method. This method allows the program to load a complete puzzle all at once.

It first accepts the new puzzle, it carefully checks if the given array has the exact right size and if all the numbers inside it are legal Sudoku numbers. Once it is sure that everything is correct, it copies all the numbers over to our actual puzzle board one by one.

```java
public void readInPuzzle(int[][] p) {
    // Validate array dimensions
    if (p == null || p.length != size * size || p[0].length != size * size) {
        return;
    }

    // Check if all values in the input puzzle are valid (0-9)
    for (int i = 0; i < p.length; i++) {
        for (int j = 0; j < p[i].length; j++) {
            if (p[i][j] < 0 || p[i][j] > 9) {
                return;
            }
        }
    }

    // Create a deep copy to ensure the original input array is not modified
    for (int i = 0; i < p.length; i++) {
        for (int j = 0; j < p[i].length; j++) {
            puzzle[i][j] = p[i][j];
        }
    }
}
```

Finding the Answer: The solve() Method

The program figures out the final answer to the puzzle. When this method starts, it first resets all the domains so the algorithm has a completely fresh start to think with.

Then, it looks at the starting numbers that are already on the board and does an initial check. It removes impossible numbers from the empty squares. After that, it starts the main Forward Checking algorithm. This algorithm is very clever. It tries to put a number in an empty square and then immediately looks ahead to see if that creates any problems or breaks any rules for the rest of the empty squares on the board. If it realizes it made a mistake, it goes back and tries a different number. If it successfully fills the whole board following all the rules, it saves the answer and tells the program that the puzzle is solved. If it tries every possible combination and finds out the puzzle is completely broken, it stops and reports that it cannot be solved.

```java
public boolean solve() {
    int n = size * size;

    // Step 1: Reset the domains to their full initial state (1-9) for a fresh solve
    D = new ArrayList<ArrayList<Integer>>(n * n);
    for (int i = 0; i < n * n; i++) {
        ArrayList<Integer> domain = new ArrayList<Integer>();
        for (int val = 1; val <= 9; val++) {
            domain.add(val);
        }
        D.add (domain);
    }

    // Step 2: Convert the current 2D board into a 1D assignment list
    ArrayList<Integer> asn = GetAssignment(puzzle);

    // Step 3: Run initial consistency check based on the starting numbers
    if (!INITIAL_FC(asn)) {
        return false; // Puzzle is unsolvable from the start
    }

    // Step 4: Run the main Forward Checking search algorithm
    ArrayList<Integer> result = FC(asn);
    if (result == null) {
        return false; // Search failed to find a valid solution
    }

    // Step 5: Convert the successful 1D result back to the 2D puzzle board
    puzzle = GetPuzzle(result);
    return true;
}
```